

Explain Promises Async Await



tell me about Promises & async/await in details with example in javascript

Absolutely. **Promises** and **async/await** are two of the most important concepts in modern JavaScript, especially for **API calls, database operations, file handling, timers, and backend development with Node.js**.

I'll explain them from beginner to interview level.

1. Why do we need Promises?

JavaScript is generally **synchronous**, meaning code executes one statement after another.

```
JavaScript
console.log("Start");

console.log("Hello");

console.log("End");
```

Output:

```
Start
Hello
End
```

But many operations take time:

- Fetching data from an API
- Reading a file
- Querying a database
- Sending a request to a server
- Waiting for a timer
- Uploading a file

For example:

```
JavaScript
console.log("Start");

setTimeout(() => {
  console.log("Data received");
}, 2000);

console.log("End");
```

Output:

Start
End
Data received

JavaScript doesn't stop and wait for the 2 seconds. The operation happens **asynchronously**. Promises provide a clean way to handle these asynchronous operations.

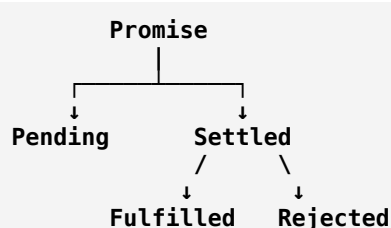
2. What is a Promise?

A **Promise** is an object that represents the eventual result of an asynchronous operation.

In simple words:

A Promise says: "I don't have the result right now, but I will give you the result later."

A Promise has three states:



1. Pending

Operation is still running.

2. Fulfilled

Operation completed successfully.

3. Rejected

Operation failed.

3. Creating a Promise

We create a Promise using the **Promise** constructor.

```

JavaScript
const promise = new Promise((resolve, reject) => {
  // asynchronous operation
});
  
```

resolve means success.

reject means failure.

Example:

```
JavaScript
const promise = new Promise((resolve, reject) => {
  let success = true;
  if (success) {
    resolve("Operation successful");
  } else {
    reject("Operation failed");
  }
});
```

Here:

```
JavaScript
resolve("Operation successful");
```

means the Promise was successful.

And:

```
JavaScript
reject("Operation failed");
```

means the Promise failed.

4. Consuming a Promise with `.then()`

We use `.then()` to handle a successful Promise.

```
JavaScript
const promise = new Promise((resolve, reject) => {
  resolve("Data received");
});

promise.then((result) => {
  console.log(result);
});
```

Output:

```
Data received
```

The value passed to:

```
JavaScript
resolve("Data received");
```

becomes the value received by:

```
JavaScript
.then((result) => {})
```

5. Handling errors with .catch()

For rejected Promises, we use `.catch()`.

```
JavaScript
const promise = new Promise((resolve, reject) => {
    reject("Something went wrong");
});

promise
    .then((result) => {
        console.log(result);
    })
    .catch((error) => {
        console.log(error);
    });
```

Output:

```
Something went wrong
```

So:

```
JavaScript
resolve()
```

→ `.then()`

and:

```
JavaScript
reject()
```

→ `.catch()`

6. .finally()

`.finally()` executes regardless of whether the Promise succeeds or fails.

```
JavaScript
const promise = new Promise((resolve, reject) => {
    resolve("Success");
});

promise
    .then((result) => {
        console.log(result);
    })
    .catch((error) => {
        console.log(error);
    })
    .finally(() => {
        console.log("Operation completed");
    });
```

Output:

```
Success
Operation completed
```

For example, this is useful for hiding a loading spinner:

```
JavaScript
fetchData()
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log(error);
  })
  .finally(() => {
    hideLoadingSpinner();
  });
```

7. Promise with setTimeout

Let's create a realistic example.

```
JavaScript
function getData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve("Data received from server");
    }, 2000);
  });
}

getData()
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log(error);
  });
```

After 2 seconds:

```
Data received from server
```

Notice something important:

```
JavaScript
function getData() {
  return new Promise(...)
}
```

The function returns a **Promise**.

8. Promise chaining

One of the most powerful features of Promises is **chaining**.

Example:

```
JavaScript
Promise.resolve(10)
  .then(value => {
    return value * 2;
  })
  .then(value => {
    return value + 5;
  })
  .then(value => {
    console.log(value);
  });
```

Output:

```
25
```

Here's what happens:

```
10
↓
10 × 2
↓
20
↓
20 + 5
↓
25
```

Each `.then()` receives the value returned by the previous `.then()`.

9. Real-world Promise chaining

Suppose:

1. Get user
2. Get user's orders
3. Get order details

You could write:

```
JavaScript
getUser()
  .then(user => {
    return getOrders(user.id);
  })
  .then(orders => {
    return getOrderDetails(orders[0].id);
  })
  .then(details => {
    console.log(details);
  })
  .catch(error => {
    console.log(error);
  });
```

This works, but as the number of operations increases, chaining can become difficult to read.

That's where `async/await` comes in.

10. What is `async/await`?

`async/await` is a cleaner way of working with Promises.

Instead of:

```
JavaScript
getUser()
  .then(user => {
    return getOrders(user.id);
  })
  .then(orders => {
    return getOrderDetails(orders[0].id);
  });
```

We can write:

```
JavaScript
async function getData() {
  const user = await getUser();
  const orders = await getOrders(user.id);
  const details = await getOrderDetails(orders[0].id);
  console.log(details);
}
```

Much easier to read.

11. The `async` keyword

When you put `async` before a function:

```
JavaScript
async function hello() {
  return "Hello";
}
```

The function **always returns a Promise**.

So:

```
JavaScript
const result = hello();

console.log(result);
```

will give a Promise.

You can handle it with:

```
JavaScript
hello().then(result => {
  console.log(result);
});
```

Output:

```
Hello
```

Even though we returned a normal string:

```
JavaScript
return "Hello";
```

JavaScript automatically wraps it in a Promise.

Conceptually:

```
JavaScript
async function hello() {
  return "Hello";
}
```

is similar to:

```
JavaScript
function hello() {
  return Promise.resolve("Hello");
}
```

12. The await keyword

await is used to wait for a Promise to settle.

Example:

```
JavaScript
function getData() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve("Data received");
    }, 2000);
  });
}

async function main() {
  const data = await getData();
  console.log(data);
}

main();
```


Output after 2 seconds:

Data received

The important part:

```
JavaScript
const data = await getData();
```

JavaScript waits for the Promise to settle before assigning its resolved value to **data**.

13. await does NOT block the entire JavaScript program

This is an important interview concept.

Consider:

```
JavaScript
async function main() {
  console.log("A");
  const data = await getData();
  console.log("B");
}
main();
console.log("C");
```

Output:

```
A
C
B
```

Why?

When JavaScript reaches:

```
JavaScript
await getData();
```

the **main()** function pauses until the Promise resolves.

But JavaScript can continue executing other work.

So **await** does **not** block the entire JavaScript thread.

14. Error handling with async/await

With **.then()**:

```
JavaScript
getData()
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log(error);
  });
```

With `async/await`, we normally use:

```
JavaScript
try...catch
```

Example:

```
JavaScript
async function main() {
  try {
    const data = await getData();
    console.log(data);
  } catch (error) {
    console.log(error);
  }
}
```

This is one of the most common patterns you'll use in real applications.

15. Complete example

Let's create a Promise that can succeed or fail.

```
JavaScript
function loginUser(username, password) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (username === "admin" && password === "1234") {
        resolve({
          id: 1,
          username: "admin"
        });
      } else {
        reject("Invalid username or password");
      }
    }, 1000);
  });
}
```

```
}
```

Using `.then()`:

JavaScript

```
loginUser("admin", "1234")
  .then(user => {
    console.log("Logged in:", user);
  })
  .catch(error => {
    console.log("Error:", error);
  });
```

Using `async/await`:

JavaScript

```
async function login() {
  try {
    const user = await loginUser("admin", "1234");
    console.log("Logged in:", user);
  } catch (error) {
    console.log("Error:", error);
  }
}

login();
```

16. Fetch API and Promises

Promises become extremely important when working with APIs.

For example:

JavaScript

```
fetch("https://api.example.com/users")
```

`fetch()` returns a Promise.

You can use:

JavaScript

```
fetch("https://api.example.com/users")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log(error);
  });
```

Or using `async/await`:

```
JavaScript
async function getUsers() {
  try {
    const response = await fetch(
      "https://api.example.com/users"
    );
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log(error);
  }
}
getUsers();
```

The second version is generally easier to maintain.

17. Important: `response.json()` is also asynchronous

This often confuses beginners.

Consider:

```
JavaScript
const response = await fetch(url);
```

This gives you the HTTP response.

Then:

```
JavaScript
const data = await response.json();
```

parses the response body.

Why another `await`?

Because:

```
JavaScript
response.json()
```

also returns a Promise.

So:

```
JavaScript
fetch(url)
```

→ Promise

and:

```
JavaScript
response.json()
```

→ Promise

18. Sequential execution

Suppose you have three asynchronous operations:

```
JavaScript
const a = await taskA();
const b = await taskB();
const c = await taskC();
```

They execute sequentially:

```
taskA
  ↓
taskB
  ↓
taskC
```

If each takes 2 seconds:

$$2s + 2s + 2s = 6s$$

This is appropriate when one task depends on the previous task.

Example:

```
JavaScript
const user = await getUser();
const orders = await getOrders(user.id);
```

You can't get the orders until you know the user's ID.

19. Parallel execution with Promise.all()

But what if tasks don't depend on each other?

For example:

```
JavaScript
const users = await getUsers();
const products = await getProducts();
const orders = await getOrders();
```

If each takes 2 seconds, sequential execution takes roughly:

6 seconds

Instead, use:

```
JavaScript
const [users, products, orders] = await Promise.all([
  getUsers(),
  getProducts(),
  getOrders()
]);
```

Now they can run concurrently.

Roughly:

```
getUsers()
getProducts()
getOrders() → results
```



Total time can be roughly:

2 seconds

instead of:

6 seconds

20. Promise.all()

`Promise.all()` waits for multiple Promises.

Example:

```
JavaScript
const promise1 = Promise.resolve(10);
const promise2 = Promise.resolve(20);
const promise3 = Promise.resolve(30);

const result = await Promise.all([
  promise1,
  promise2,
  promise3
]);

console.log(result);
```

Output:

```
JavaScript
[10, 20, 30]
```

Notice that the order is preserved.

Even if the Promises finish in a different order, the results correspond to the original array order.

21. What happens if one Promise fails?

With:

```
JavaScript
Promise.all([
  promise1,
  promise2,
  promise3
]);
```

if one rejects, `Promise.all()` rejects.

Example:

```
JavaScript
const p1 = Promise.resolve("A");
const p2 = Promise.reject("Something failed");
const p3 = Promise.resolve("C");
try {
  const result = await Promise.all([
    p1,
    p2,
    p3
  ]);
  console.log(result);
} catch (error) {
  console.log(error);
}
```

Output:

```
Something failed
```

22. Promise.allSettled()

Sometimes you don't want one failure to affect the others.

Use:

```
JavaScript
Promise.allSettled()
```

Example:

```
JavaScript
const results = await Promise.allSettled([
  Promise.resolve("Success"),
  Promise.reject("Failed"),
  Promise.resolve("Another success")
]);
```

```
console.log(results);
```

You'll get results describing each Promise:

```
JavaScript
[
  { status: "fulfilled", value: "Success" },
  { status: "rejected", reason: "Failed" },
  { status: "fulfilled", value: "Another success" }
]
```

Useful when you want the outcome of **every operation**, regardless of failures.

23. Promise.race()

`Promise.race()` returns the result of whichever Promise settles first.

Example:

```
JavaScript
const p1 = new Promise(resolve => {
  setTimeout(() => resolve("First"), 1000);
});

const p2 = new Promise(resolve => {
  setTimeout(() => resolve("Second"), 2000);
});

const result = await Promise.race([p1, p2]);

console.log(result);
```

Output:

```
First
```

because `p1` finishes first.

24. Promise.any()

`Promise.any()` returns the **first fulfilled Promise**.

Example:

```
JavaScript
const p1 = Promise.reject("Server 1 failed");

const p2 = new Promise(resolve => {
  setTimeout(() => resolve("Server 2 worked"), 1000);
});

const p3 = new Promise(resolve => {
  setTimeout(() => resolve("Server 3 worked"), 2000);
});

const result = await Promise.any([
  p1,
```



```

    p2,
    p3
  });

  console.log(result);

```

Output:

```
Server 2 worked
```

It ignores rejected Promises until it finds a successful one.

25. Promise methods comparison

Method	Behavior
<code>Promise.all()</code>	Wait for all; fails if one rejects
<code>Promise.allSettled()</code>	Wait for all regardless of success/failure
<code>Promise.race()</code>	First Promise to settle wins
<code>Promise.any()</code>	First Promise to fulfill wins

A simple way to remember:

```

all
↓
Everyone must succeed

allSettled
↓
Tell me everyone's result

race
↓
Who finishes first?

any
↓
Who succeeds first?

```

26. Callback vs Promise vs async/await

Callback

```

JavaScript
getUser(function(user) {
  getOrders(user.id, function(orders) {
    getOrderDetails(orders[0], function(details) {
      console.log(details);
    });
  });
});

```

This can lead to **callback hell**.

Promise

```
JavaScript
getUser()
  .then(user => getOrders(user.id))
  .then(orders => getOrderDetails(orders[0]))
  .then(details => {
    console.log(details);
  })
  .catch(error => {
    console.log(error);
  });
```

Better.

Async/await

```
JavaScript
async function main() {
  try {
    const user = await getUser();
    const orders = await getOrders(user.id);
    const details = await getOrderDetails(orders[0]);
    console.log(details);
  } catch (error) {
    console.log(error);
  }
}
```

Usually the easiest to read.

27. Common mistake: forgetting await

Consider:

```
JavaScript
async function getData() {
  return "Hello";
}

async function main() {
  const data = getData();
  console.log(data);
}

main();
```

`data` is a Promise, not "Hello".

You need:

```
JavaScript
const data = await getData();
```

So:

```
JavaScript
async function main() {
  const data = await getData();
  console.log(data);
}
```

Output:

```
Hello
```

28. Common mistake: forgetting return

Consider:

```
JavaScript
function getData() {
  fetch("/api/users")
    .then(response => response.json());
}
```

The function doesn't return the Promise.

Better:

```
JavaScript
function getData() {
  return fetch("/api/users")
    .then(response => response.json());
}
```

Or:

```
JavaScript
async function getData() {
  const response = await fetch("/api/users");
  return response.json();
}
```

29. async functions always return Promises

This is very important.

```
JavaScript
async function test() {
  return 100;
}
```

You can think of it as:

```
JavaScript
function test() {
  return Promise.resolve(100);
}
```

Therefore:

```
JavaScript
test().then(value => {
  console.log(value);
});
```

prints:

```
100
```

30. await works with non-Promises too

This is valid:

```
JavaScript
async function test() {
  const value = await 10;
  console.log(value);
}
test();
```

Output:

```
10
```

JavaScript effectively treats the value as an already-resolved Promise.

But **await** is mainly useful with asynchronous operations.

31. async/await with Express

Since you're learning backend development with Node.js, this is particularly important.

Suppose you have an Express route:

```
JavaScript
app.get("/users", async (req, res) => {
  try {
    const users = await User.find();
    res.json(users);
  } catch (error) {
    res.status(500).json({
      message: "Server error"
    });
  }
});
```

Here:

```
JavaScript
User.find()
```

returns a Promise.

So we use:

```
JavaScript
await User.find();
```

This pattern is extremely common in Node.js applications.

32. Real-world API example

Imagine your frontend sends:

```
GET /api/products
```

Your backend:

```
JavaScript
app.get("/api/products", async (req, res) => {
  try {
    const products = await Product.find();
    res.status(200).json(products);
  } catch (error) {
    console.error(error);
    res.status(500).json({
      message: "Failed to fetch products"
    });
  }
});
```

```
}  
});
```

The flow is:

```
Frontend  
↓  
HTTP Request  
↓  
Express Route  
↓  
Product.find()  
↓  
Database  
↓  
Promise resolves  
↓  
products  
↓  
JSON Response  
↓  
Frontend
```

This is exactly where understanding Promises and **async/await** becomes useful in full-stack development.

33. Async/Await + Axios

You'll also frequently encounter this pattern:

```
JavaScript  
async function getProducts() {  
  try {  
    const response = await axios.get("/api/products");  
    console.log(response.data);  
  } catch (error) {  
    console.log(error);  
  }  
}
```

And POST requests:

```
JavaScript  
async function createProduct(product) {  
  try {  
    const response = await axios.post(  
      "/api/products",  
      product  
    );  
    console.log(response.data);  
  } catch (error) {  
    console.log(error);  
  }  
}
```

```
}  
}
```

34. Promise error propagation

Errors can move through a Promise chain.

```
JavaScript  
function test() {  
    return Promise.reject("Something went wrong");  
}  
  
test()  
    .then(result => {  
        console.log(result);  
    })  
    .catch(error => {  
        console.log(error);  
    });
```

Output:

```
Something went wrong
```

Similarly:

```
JavaScript  
async function test() {  
    throw new Error("Something went wrong");  
}
```

An error thrown inside an **async** function causes the returned Promise to reject.

So:

```
JavaScript  
try {  
    await test();  
} catch (error) {  
    console.log(error.message);  
}
```

35. throw VS reject

You may see both:

```
JavaScript  
reject(new Error("Failed"));
```

and:

```
JavaScript
throw new Error("Failed");
```

With async/await:

```
JavaScript
async function test() {
    throw new Error("Failed");
}
```

With a manually created Promise:

```
JavaScript
const promise = new Promise((resolve, reject) => {
    reject(new Error("Failed"));
});
```

Both ultimately result in a rejected Promise.

36. Important interview question: Promise vs async/await

Promise

Promise is an object representing the eventual completion/failure of an asynchronous operation.

Example:

```
JavaScript
fetch(url)
    .then(...)
    .catch(...);
```

async/await

async/await is syntax built around Promises that makes asynchronous code easier to read.

Example:

```
JavaScript
try {
    const response = await fetch(url);
} catch (error) {
    console.log(error);
}
```

Important: **async/await** does not replace Promises. It **uses Promises**.

Think:

```
Promises
  ↓
Foundation
  ↓
async/await
  ↓
Cleaner syntax for working with Promises
```

37. Promise execution example

This is a very common interview question.

What is the output?

```
JavaScript
console.log("A");

Promise.resolve().then(() => {
  console.log("B");
});

console.log("C");
```

Output:

```
A
C
B
```

Why?

Promise callbacks run through the **microtask queue**.

Simplified:

```
Call Stack
  ↓
Synchronous code
  ↓
Microtask Queue
  ↓
Promise .then()
```

38. Another interview example

```
JavaScript
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

Output:

```
Start
End
Promise
Timeout
```

Why?

Simplified event-loop ordering:

```
Synchronous code
↓
Microtasks
↓
Macrotasks
```

Promises use the **microtask queue**, while `setTimeout` callbacks go through the task/macrotask queue.

39. A practical example: user dashboard

Imagine your dashboard needs:

```
User
Products
Notifications
```

These don't depend on each other.

Bad/sequential approach:

```
JavaScript
const user = await getUser();
const products = await getProducts();
const notifications = await getNotifications();
```

Better:

```
JavaScript
const [
  user,
  products,
  notifications
] = await Promise.all([
  getUser(),
  getProducts(),
  getNotifications()
]);
```

This can significantly reduce waiting time.

40. When should you use sequential `await`?

Use:

JavaScript

```
const user = await getUser();
const orders = await getOrders(user.id);
```

when the second operation depends on the first.

```
getUser()
  ↓
user.id
  ↓
getOrders(user.id)
```

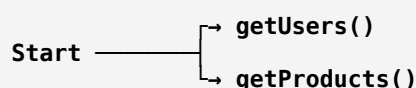
41. When should you use `Promise.all()`?

Use:

JavaScript

```
const [users, products] = await Promise.all([
  getUsers(),
  getProducts()
]);
```

when operations are independent.



This is an important performance optimization.

42. The most important syntax to remember

Creating a Promise

JavaScript

```
const promise = new Promise((resolve, reject) => {
  // operation
  resolve(value);
  // or
  reject(error);
});
```

Consuming a Promise

JavaScript

```
promise
  .then(result => {
    console.log(result);
  })
  .catch(error => {
    console.log(error);
  });
```

```

    })
    .finally(() => {
      console.log("Done");
    });
  });

```

Async/await

```

JavaScript
async function main() {
  try {
    const result = await promise;
    console.log(result);
  } catch (error) {
    console.log(error);
  }
}

```

Parallel operations

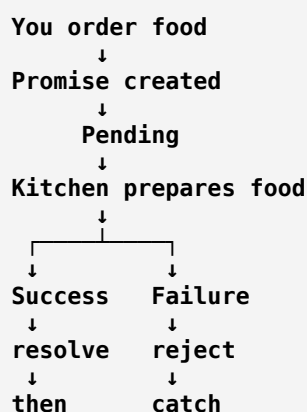
```

JavaScript
const [a, b, c] = await Promise.all([
  taskA(),
  taskB(),
  taskC()
]);

```

43. Promise mental model

Think of a Promise like ordering food at a restaurant.



`await` is basically:

"I'll continue this function once my order is ready."

44. What you should learn next

Since you're learning JavaScript systematically, I'd learn these in this order:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

1. Synchronous vs asynchronous JavaScript



2. Callbacks



3. Callback Hell



4. Promises



5. .then()



6. .catch()



7. .finally()



8. Promise chaining



9. async



10. await



11. try/catch



12. Promise.all()



13. Promise.allSettled()



14. Promise.race()



15. Promise.any()



16. Event Loop + Microtask Queue



17. Fetch/Axios



18. Async/Await in Node.js + Express

The key takeaway

If you remember only one thing:

JavaScript

// Promise style

```
getData()
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log(error);
  });
```

is essentially written more cleanly as:

JavaScript

```
async function main() {
  try {
    const data = await getData();
    console.log(data);
  } catch (error) {
    console.log(error);
  }
}
```

Promises are the underlying mechanism; `async/await` is the cleaner syntax for working with them.

ChatGPT can make mistakes. Check important info.